

Functions in C

A **function** is a named block of C statements designed to perform a particular task. Functions help us divide a large program into **small, manageable, reusable modules**.

It is also defined as: A function is a named block of code that performs a specific task. It helps divide a large program into smaller and reusable parts.

For example, instead of writing the code for addition every time we need it, we can create an add() function and call it whenever required.

```
int add(int a, int b)
{
    return a + b;
}
```

1. Introduction to Functions

Consider this program:

```
#include <stdio.h>
int main()
{
    int a = 10, b = 20;
    int sum;
    sum = a + b;
    printf("Sum = %d", sum);
    return 0;
}
```

Here, everything is written inside main().

As a program becomes larger, putting everything inside main() makes it difficult to understand and maintain.

We can divide it into functions:

```
#include <stdio.h>
int add(int a, int b)
{
    return a + b;
}
int main()
{
    int result;
    result = add(10, 20);
    printf("Sum = %d", result);
    return 0;
}
```

Advantages of functions

Advantage	Meaning
Modularity	Divides a large program into smaller parts
Reusability	Same function can be called many times

Advantage	Meaning
Readability	Program becomes easier to understand
Debugging	Errors can be isolated more easily
Maintenance	Changes can be made in one place
Avoid repetition	Avoids writing the same code repeatedly

2. Types of Functions

Functions in C can broadly be divided into two categories.

A. Library functions

These are functions already provided by C libraries.

Examples:

printf()

scanf()

strlen()

sqrt()

strcmp()

For example:

```
#include <stdio.h>
#include <math.h>
int main()
{
    printf("Square root = %.2f", sqrt(25));
    return 0;
}
```

Output:

Square root = 5.00

Here:

- printf() comes from <stdio.h>
- sqrt() comes from <math.h>

B. User-defined functions

These are functions created by the programmer.

Example:

```
int square(int n)
{
    return n * n;
}
```

Then:

```
printf("%d", square(5));
```

Output:

25

3. Four Common Types of User-Defined Functions

User-defined functions are often classified according to whether they **take arguments** and whether they **return a value**.

Type	Arguments	Return value
1	No	No
2	Yes	No
3	No	Yes
4	Yes	Yes

Let's understand each.

Type 1: No arguments and no return value

```
#include <stdio.h>
```

```
void message()
```

```
{
```

```
    printf("Hello from function!");
```

```
}
```

```
int main()
```

```
{
```

```
    message();
```

```
    return 0;
```

```
}
```

Output:

Hello from function!

Here:

```
void message()
```

means the function doesn't return a value.

And:

```
message();
```

calls the function.

Type 2: Arguments but no return value

```
#include <stdio.h>
```

```
void display(int n)
```

```
{
```

```
    printf("Number = %d", n);
```

```
}
```

```
int main()
```

```
{
```

```
    display(10);
```

```
    return 0;
```

```
}
```

Output:

Number = 10

Here 10 is passed to the function.

Type 3: No arguments but returns a value

```
#include <stdio.h>
```

```
int getNumber()
```

```
{
```

```
    return 100;
```

```
}
```

```
int main()
```

```
{
```

```
    int n;
```

```
    n = getNumber();
```

```
    printf("Number = %d", n);
```

```
    return 0;
```

```
}
```

Output:

Number = 100

Type 4: Arguments and return value

This is probably the most commonly used form.

```
#include <stdio.h>
```

```
int add(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

```
int main()
```

```
{
```

```
    int result;
```

```
    result = add(10, 20);
```

```
    printf("Sum = %d", result);
```

```
    return 0;
```

```
}
```

Output:

Sum = 30

Here:

10, 20

↓ ↓

arguments

are passed to:

a, b

and:

return a + b;

returns 30.

4. Declaration of a Function

A function declaration tells the compiler:

"A function with this name exists, and this is its return type and parameter information."

For example:

```
int add(int, int);
```

This is a **function declaration/prototype**.

It tells the compiler:

- function name = add
- return type = int
- number of parameters = 2
- parameter types = int, int

The actual implementation comes later:

```
int add(int a, int b)
{
    return a + b;
}
```

5. Function Prototype

A **function prototype** is a declaration that specifies the function's interface.

Example:

```
int multiply(int, int);
```

or:

```
int multiply(int a, int b);
```

Both are valid prototypes.

Why do we need a prototype?

Suppose the function is defined **after main()**:

```
#include <stdio.h>
```

```
int main()
```

```
{
    int result;
    result = add(10, 20);
    printf("%d", result);
    return 0;
}
```

```
int add(int a, int b)
{
    return a + b;
}
```

The compiler encounters:

```
add(10, 20);
```

before it has seen the definition of add().

Therefore, we normally provide a prototype:

```
#include <stdio.h>
```

```
int add(int, int);    // Function prototype
```

```
int main()
```

```
{
```

```
    int result;
```

```
    result = add(10, 20);
```

```
    printf("%d", result);
```

```
    return 0;
```

```
}
```

```
int add(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

Now the compiler already knows about add().

6. Function Call

A **function call** is the statement/expression used to execute a function.

Suppose:

```
int add(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

Calling it:

```
add(10, 20);
```

is a function call.

If we want to store its returned value:

```
int result;
```

```
result = add(10, 20);
```

The sequence is:

```
main()
```

```
|
```

```
| add(10,20)
```

```
↓
```

```
add()
```

```
|
```

```
| return 30
```

```
↓
```

```
main()
```

```
|
```

```
↓
```

```
result = 30
```

7. Defining a Function

A function definition contains the **actual code** that performs the task.

General syntax:

```
return_type function_name(parameter_list)
{
    // statements
}
```

Example:

```
int add(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```

It has four important parts:

```
int    add    (int a, int b)
  ↑      ↑      ↑
return function parameters
type   name
```

and:

```
{
    ...
    return sum;
}
```

contains the function body.

8. Passing Arguments to a Function

Arguments are values supplied when calling a function.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Call:

```
add(10, 20);
```

Here:

- 10 → argument
- 20 → argument
- a → parameter
- b → parameter

Parameter vs Argument

Term	Example
Parameter	int a, int b
Argument	10, 20

Think of it like:

Function definition:

```
add(int a, int b)
```

↑ ↑

parameters

Function call:

```
add(10, 20)
```

↑ ↑

arguments

9. Return Values and Their Types

A function can return a value using the return statement.

Example:

```
int square(int n)
```

```
{
```

```
    return n * n;
```

```
}
```

Calling:

```
int result = square(5);
```

gives:

```
result = 25
```

The return type is:

```
int
```

because the function returns an integer.

Different return types

Integer

```
int add(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

Floating point

```
float average(float a, float b)
```

```
{
```

```
    return (a + b) / 2;
```

```
}
```

Character

```
char getGrade()
```

```
{
```

```
    return 'A';
```

```
}
```

No return value

```
void display()
```

```
{
```

```
    printf("Hello");
```

```
}
```

A void function doesn't return a value.

10. return Statement

The return statement does two things:

1. It terminates the function.
2. It can send a value back to the calling function.

Example:

```
int maximum(int a, int b)
{
    if (a > b)
        return a;
    return b;
}
```

If we call:

maximum(10, 20)

the function returns:

20

11. Multifunction Program

A program containing several functions is called a **multifunction program**.

For example, instead of doing everything in main():

```
int main()
{
    // input
    // calculation
    // display
}
```

we can divide it:

```
main()
├── input()
├── calculate()
└── display()
```

Example:

```
#include <stdio.h>
```

```
int getNumber()
```

```
{
    int n;
    printf("Enter number: ");
    scanf("%d", &n);
    return n;
}
```

```
int square(int n)
```

```
{
    return n * n;
}
```

```
void display(int n)
```

```
{
    printf("Square = %d", n);
}
```

```

int main()
{
    int number;
    int result;
    number = getNumber();
    result = square(number);
    display(result);
    return 0;
}

```

Output:

Enter number: 7

Square = 49

This is much easier to understand because each function performs **one specific task**.

12. Calling Function by Value

This is an important concept.

In C, ordinary function arguments are passed **by value**.

That means the function receives a **copy** of the value.

Consider:

```

#include <stdio.h>
void change(int x)
{
    x = 100;
}
int main()
{
    int n = 10;
    printf("Before = %d\n", n);
    change(n);
    printf("After = %d\n", n);
    return 0;
}

```

Output:

Before = 10

After = 10

Why didn't n change?

Because:

main()	change()
-----	-----
n = 10	-----> x = 10
	x = 100

x is only a **copy**.

Changing x doesn't change n.

Important point

C uses pass-by-value for ordinary function arguments.

When pointers are used, we can modify the caller's object through its address. That is commonly described as achieving "pass-by-reference-like" behaviour, although technically C still passes the pointer itself **by value**.

13. Recursive Functions

A function that calls **itself** is called a **recursive function**.

For example:

```
void fun()
{
    fun();
}
```

This is recursion, but it has no stopping condition, so it would continue until the program runs out of stack space.

Therefore, recursion normally needs a:

Base case

The condition that stops recursion.

Example: Factorial

Mathematically:

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

We can express it recursively:

$$n! = n \times (n-1)!$$

and:

$$0! = 1$$

So:

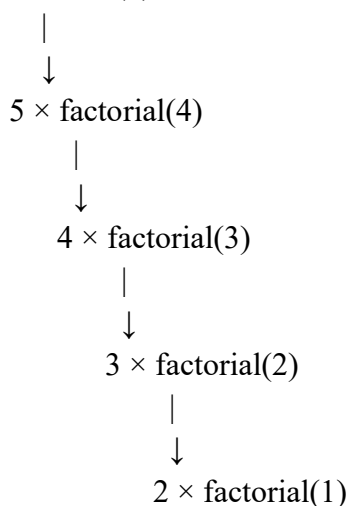
```
int factorial(int n)
{
    if (n == 0)
        return 1;
    return n * factorial(n - 1);
}
```

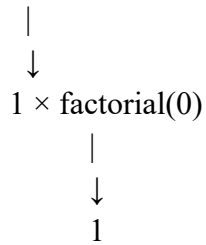
Calling:

factorial(5)

works like this:

factorial(5)





Then the results return upward:

$\text{factorial}(0) = 1$

$\text{factorial}(1) = 1 \times 1 = 1$

$\text{factorial}(2) = 2 \times 1 = 2$

$\text{factorial}(3) = 3 \times 2 = 6$

$\text{factorial}(4) = 4 \times 6 = 24$

$\text{factorial}(5) = 5 \times 24 = 120$

Output:

Factorial = 120

14. Another Recursive Example — Sum of Natural Numbers

We can calculate:

$1 + 2 + 3 + 4 + 5$

using recursion:

```

int sum(int n)
{
    if (n == 0)
        return 0;
    return n + sum(n - 1);
}

```

For:

$\text{sum}(5)$

we get:

$5 + \text{sum}(4)$

$5 + 4 + \text{sum}(3)$

$5 + 4 + 3 + \text{sum}(2)$

$5 + 4 + 3 + 2 + \text{sum}(1)$

$5 + 4 + 3 + 2 + 1 + \text{sum}(0)$

Since:

$\text{sum}(0) = 0$

the result is:

15

15. Complete Function Structure

A typical C program using functions can be organized as:

```
#include <stdio.h>
```

```
/* Function prototypes */
```

```
int add(int, int);
```

```
int square(int);
```

```
int factorial(int);
```

```

int main()
{
    int a = 10;
    int b = 20;
    printf("Sum = %d\n", add(a, b));
    printf("Square = %d\n", square(a));
    printf("Factorial = %d\n", factorial(5));
    return 0;
}

```

/* Function definitions */

```

int add(int x, int y)
{
    return x + y;
}
int square(int n)
{
    return n * n;
}
int factorial(int n)
{
    if (n == 0)
        return 1;

    return n * factorial(n - 1);
}

```

Notice the three important stages:

1. Declaration / Prototype

↓

2. Function Call

↓

3. Function Definition

Single C Program Implementing All Concepts

The following program combines **function declaration, prototype, function calls, arguments, return values, multiple functions, pass-by-value, and recursion** in one program.

```

#include <stdio.h>
/*
    Function Prototypes
    -----
    These tell the compiler about the functions
    before they are called from main().
*/
int add(int, int);
int square(int);
int factorial(int);

```

```

void displayNumber(int);
void changeValue(int);
/*
    main() is also a function.
    Program execution starts from main().
*/
int main()
{
    int a = 10;
    int b = 20;
    int result;
    int n = 5;
    printf("===== FUNCTION DEMONSTRATION =====\n\n");
    /*
        1. Function call with arguments and a return value.
        add(10, 20)
            10 and 20 -> arguments
            The returned value 30 is stored
            in result.
    */
    result = add(a, b);
    printf("1. Sum = %d\n", result);
    /*
        2. Function with one argument
        and a return value.
    */
    printf("2. Square of %d = %d\n",
        a, square(a));
    /*
        3. Function with an argument
        but no return value.
        displayNumber() simply displays
        the value.
    */
    printf("3. Displaying a number: ");
    displayNumber(a);
    /*

4. Calling function by value.
    n contains 10.
    changeValue() receives a COPY of n.
    Therefore, changing x inside the
    function will NOT change n.
    */
    printf("\n4. Before changeValue(): n = %d\n", a);
    changeValue(a);
    printf("   After changeValue(): n = %d\n", a);

```

```

/*
    5. Recursive function.
    factorial(5) calculates:
    5 * 4 * 3 * 2 * 1 = 120
*/
printf("\n5. Factorial of %d = %d\n",
    n, factorial(n));
return 0;
}
/*
    Function Definition: add()
    Takes two integer arguments
    and returns their sum.
*/
int add(int x, int y)
{
    return x + y;
}
/*
    Function Definition: square()
    Takes one integer argument
    and returns its square.
*/
int square(int n)
{
    return n * n;
}
/*
    Function Definition: displayNumber()
    Takes an integer argument but does not return any value.
    Therefore, its return type is void.
*/
void displayNumber(int n)
{
    printf("%d\n", n);
}
/*
    Function Definition: changeValue()
    This demonstrates PASSING BY VALUE.
    x receives a COPY of the value
    passed from main().
*/
void changeValue(int x)
{
    printf("    Inside changeValue(): x = %d\n", x);
}
/*
    Only the local copy is changed.
    The original variable in main()

```

```

    remains unchanged.
*/
x = 100;
printf(" Inside changeValue(): x = %d\n", x);
}
/*
Function Definition: factorial()
This is a RECURSIVE FUNCTION because it calls itself.
Base case:
    factorial(0) = 1
Recursive case:
    n! = n * (n-1)!
*/
int factorial(int n)
{
    /*
    Base case:
    This stops the recursion.
    */
    if (n == 0)
    {
        return 1;
    }
    /*
    Recursive call:
    factorial() calls itself.
    */
    return n * factorial(n - 1);
}

```

Expected Output

===== FUNCTION DEMONSTRATION =====

1. Sum = 30
2. Square of 10 = 100
3. Displaying a number: 10
4. Before changeValue(): n = 10
 Inside changeValue(): x = 10
 Inside changeValue(): x = 100
 After changeValue(): n = 10
5. Factorial of 5 = 120

Quick Revision Table

Topic	Key idea	Example
Function	Reusable block of code	add()
Library function	Provided by C libraries	printf()
User-defined function	Created by programmer	square()
Declaration	Announces function	int add(int,int);

Topic	Key idea	Example
Prototype	Function declaration with parameter information	int add(int, int);
Function call	Executes a function	add(10,20)
Definition	Contains actual function code	int add(...) { ... }
Argument	Value passed to function	10, 20
Parameter	Variable receiving argument	int a, int b
Return value	Value sent back by function	return a+b;
void	No return value	void display()
Multifunction program	Program containing several functions	main(), add(), square()
Pass by value	Function receives a copy	changeValue(a)
Recursion	Function calls itself	factorial()
Base case	Stops recursion	if(n == 0)

Important Terms:

Term	Common C terminology	Where it appears
Actual parameter	Argument	Function call
Formal parameter	Parameter	Function definition

1. Actual Parameter

An **actual parameter** is the value or variable supplied to a function **when the function is called**.

Consider:

```
#include <stdio.h>
void add(int a, int b)
{
    printf("Sum = %d", a + b);
}
int main()
{
    int x = 10;
    int y = 20;
    add(x, y);    // x and y are actual parameters
    return 0;
}
```

Here:

```
add(x, y);
```

```
  ↑ ↑
```

Actual parameters

x and y are the **actual parameters** because they are supplied during the function call.

They can also be constants:

```
add(10, 20);
```

Here 10 and 20 are actual parameters (often also called **arguments**).

Simple definition

Actual parameters are the values, variables, or expressions passed to a function during the function call.

2. Formal Parameter

A **formal parameter** is the variable declared in the function definition that receives the value passed by the caller.

Example:

```
#include <stdio.h>
void add(int a, int b)
{
    printf("Sum = %d", a + b);
}
int main()
{
    add(10, 20);
    return 0;
}
```

Look at:

```
void add(int a, int b)
```

↑ ↑

Formal parameters

a and b are **formal parameters**.

They receive the values supplied by the actual parameters:

Actual parameters Formal parameters

10 -----> a

20 -----> b

Simple definition

Formal parameters are the variables declared in a function definition that receive values from the actual parameters.

3. Actual vs Formal Parameter

Consider:

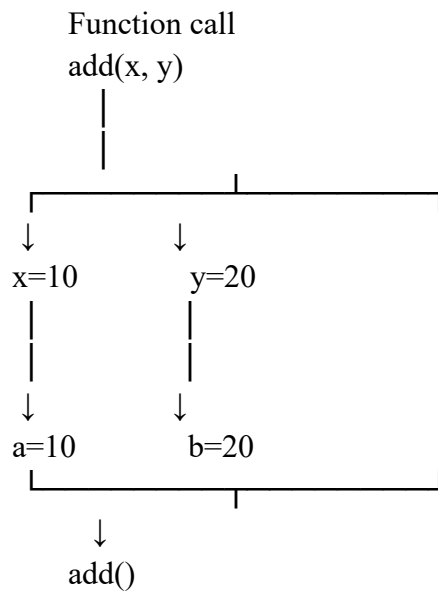
```
int add(int a, int b)
{
    return a + b;
}
int main()
{
    int x = 10;
    int y = 20;
    int result = add(x, y);
    return 0;
}
```

Here:

Item	Role
------	------

Item	Role
x, y	Actual parameters
a, b	Formal parameters
add(x, y)	Function call
int add(int a, int b)	Function definition

The relationship is:



4. Call by Value

In **call by value**, a **copy of the actual parameter's value** is passed to the function.

Therefore, changes made to the formal parameter **do not affect the original variable**.

Consider:

```

#include <stdio.h>
void change(int x)
{
    x = 100;
}
int main()
{
    int n = 10;
    printf("Before = %d\n", n);
    change(n);
    printf("After = %d\n", n);
    return 0;
}

```

Output

Before = 10

After = 10

Why?

When we write:

change(n);

the value of n is copied into x.

main() change()

$$n = 10 \quad \text{-----} \rightarrow \quad x = 10$$

$x = 100$

The two variables are separate.

n = 10 x = 10

└─── remains 10

└─── changed to 100

So after the function finishes:

$$n = 10$$

Important point

In call by value, the function works on a **copy** of the data.

5. A Better Example: Swapping Using Call by Value

Suppose we want to swap two numbers.

```
#include <stdio.h>

void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 10;
    int y = 20;
    printf("Before swap: x = %d, y = %d\n", x, y);
    swap(x, y);
    printf("After swap: x = %d, y = %d\n", x, y);
    return 0;
}
```

Output:

Before swap: $x = 10, y = 20$

After swap: $x = 10, y = 20$

The swap happens only to the **copies**.

```

main()                swap()
x = 10 -----> a = 10
y = 20 -----> b = 20
                a = 20
                b = 10
x = 10            y = 20

```

Therefore, the original values don't change.

6. Call by Reference

Here we need to make an important distinction for C.

C does NOT have a true "call by reference" mechanism.

C has **call by value**.

However, we can achieve **reference-like behaviour by passing addresses using pointers**.

This is commonly called **call by reference in C textbooks**, although technically it is **passing a pointer by value**.

7. Call by Reference Using Pointers

Let's modify the previous example.

```
#include <stdio.h>
void change(int *x)
{
    *x = 100;
}
int main()
{
    int n = 10;
    printf("Before = %d\n", n);
    change(&n);
    printf("After = %d\n", n);
    return 0;
}
```

Output:

Before = 10

After = 100

Now the original variable changes.

Why?

We pass its **address**:

change(&n);

&n means:

Address of n

The function receives that address:

void change(int *x)

So x contains the address of n.

Then:

*x = 100;

means:

Go to the address stored in x and change the value there to 100.

Conceptually:

main() change()

n = 10

Address = 1000

↓
|
| &n
↓

x = 1000

$\downarrow$
 $*x$
 $\downarrow$
 $n = 100$

Therefore, the original n is changed.

8. Call by Reference — Swap Example

This is the classic example.

```
#include <stdio.h>
void swap(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
int main()
{
    int x = 10;
    int y = 20;
    printf("Before swap: x = %d, y = %d\n", x, y);
    swap(&x, &y);
    printf("After swap: x = %d, y = %d\n", x, y);
    return 0;
}
```

Output

Before swap: x = 10, y = 20

After swap: x = 20, y = 10

Let's understand this carefully.

Function call

swap(&x, &y);

We pass:

&x → address of x

&y → address of y

Function parameters

void swap(int *a, int *b)

Therefore:

a → address of x

b → address of y

Inside function

temp = *a;

*a means:

value stored at the address contained in a

So:

*a = 10

*b = 20

Then:

```

*a = *b;
makes:
x = 20
and:
*b = temp;
makes:
y = 10

```

9. Call by Value vs Reference-like Calling

Feature	Call by Value	Reference-like calling in C
What is passed?	Value	Address
Function parameter	Normal variable	Pointer
Example call	change(n)	change(&n)
Example parameter	int x	int *x
Can modify original variable?	No	Yes
Uses pointer?	No	Yes
Technically C mechanism	Pass by value	Pointer passed by value

10. Very Important: C's Technical Terminology

Suppose we have:

```

void change(int *p)
{
    *p = 100;
}
int main()
{
    int x = 10;
    change(&x);
    return 0;
}

```

It is commonly described as:

Call by reference using pointers

But technically:

C always passes function arguments by value.

What is passed by value here?

The **address** is passed by value.

```

x
|
| address = 5000
↓
&x = 5000
|
↓
p = 5000
p itself is a copy of the address.

```

So:

x's address p's value
5000 → 5000

Both point to the same memory location.

That's why:

*p = 100;

changes x.

11. One Program Demonstrating Everything

```
#include <stdio.h>
/* Function prototypes */
void changeValue(int x);
void changeReference(int *x);
void swapValue(int a, int b);
void swapReference(int *a, int *b);
int main()
{
    int n = 10;
    int x = 10;
    int y = 20;
    /*
        x and y in this function call
        are ACTUAL PARAMETERS.
    */
    printf("Original n = %d\n", n);
    /*
        Call by value:
        A copy of n is passed.
    */
    changeValue(n);
    printf("After call by value = %d\n", n);
    /*
        Call by reference (reference-like):
        Address of n is passed.
    */
    changeReference(&n);
    printf("After reference-like call = %d\n", n);
    printf("\nBefore swap by value:\n");
    printf("x = %d, y = %d\n", x, y);
    swapValue(x, y);
    printf("After swap by value:\n");
    printf("x = %d, y = %d\n", x, y);
    printf("\nBefore swap by reference:\n");
    printf("x = %d, y = %d\n", x, y);
    swapReference(&x, &y);
    printf("After swap by reference:\n");
    printf("x = %d, y = %d\n", x, y);
```

```

    return 0;
}
/*
    x is a FORMAL PARAMETER.
    The value of n is copied into x.
*/
void changeValue(int x)
{
    x = 100;
    printf("Inside changeValue(): x = %d\n", x);
}
/*
    x is a FORMAL PARAMETER.
    It is a pointer containing the
    address of the actual variable.
*/
void changeReference(int *x)
{
    *x = 100;
}

/*
    Call by value.
    a and b are copies of x and y.
*/
void swapValue(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
/*
    Reference-like behavior using pointers.
    a contains the address of x.
    b contains the address of y.
*/
void swapReference(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}

```

Output

Original n = 10

Inside changeValue(): x = 100

After call by value = 10

After reference-like call = 100

Before swap by value:

x = 10, y = 20

After swap by value:

x = 10, y = 20

Before swap by reference:

x = 10, y = 20

After swap by reference:

x = 20, y = 10

One-line definitions:

- **Actual parameter:** The value, variable, or expression supplied to a function during the function call.
 - **Formal parameter:** The variable declared in the function definition that receives the passed value.
 - **Call by value:** A copy of the actual parameter's value is passed to the function, so changes to the formal parameter do not affect the original.
 - **Call by reference:** In common C terminology, an address is passed using a pointer so the function can modify the original variable. **Technically, C passes that pointer itself by value; C has no true pass-by-reference parameter mechanism.**
-